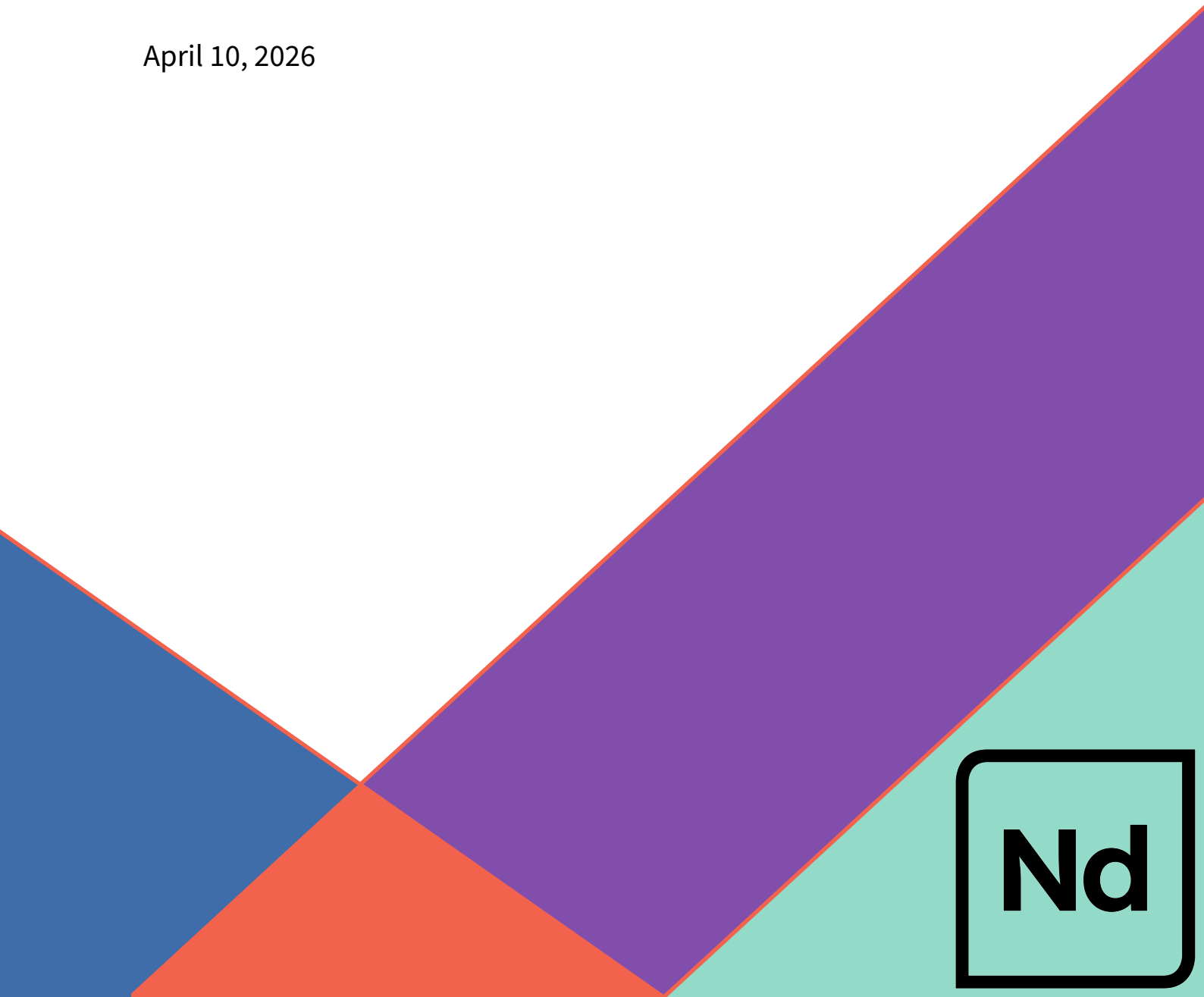

Change Review - Stake Program v5.0.0

conducted by Neodyme AG

Lead Auditor:	Sebastian Fritsch
Administrative Lead:	Thomas Lambertz

April 10, 2026



Nd

Table of Contents

1	Introduction	3
2	Scope	3
3	Project Overview	4
	Minimum Delegation	4
	Rent Sysvar	4
	Optional Sysvar Accounts	4
	Rewrite of the Split Instruction	4
	Rent Change Interactions	5
4	Findings	6
Appendices		
A	About Neodyme	7
B	Methodology	8
	Select Common Vulnerabilities	8
C	Vulnerability Severity Rating	10

1 | Introduction

From March 2026 until April 2026, Anza engaged Neodyme to do a change review of the stake program v5.0.0. The review focuses on the changes between v4.0.0 and v5.0.0. Especially the following changes were reviewed:

- Introduction of a minimum delegation of 1 SOL
- Relying on the Rent sysvar instead of `meta.rent_exempt_reserve` for rent calculations
- Sysvar accounts are made optional, instead relying on syscalls
- Rewrite of the `Split` instruction

Additionally the program was reviewed for interaction with potential rent decrease ([SIMD-0436](#)) and increase proposals ([SIMD-0392](#)).

2 | Scope

Neodyme considers the source code, located at <https://github.com/solana-program/stake>, in scope for this audit. Third-party dependencies are not in scope.

Relevant source code revisions are:

- `c7dd704881161795b979d8038cbb7a291abe6b12` · v4.0.0 · Diff start
- `d27f520a97f3199b251a19be958cfb0d185e5f84` · Last reviewed revision

3 | Project Overview

This section briefly outlines the reviewed changes to the stake program and discusses interactions with potential rent changes.

Minimum Delegation

The minimum delegation is raised from 1 lamport to 1 SOL via `get_minimum_delegation()`. This affects delegation, split, and move operations. Accounts with active delegation below 1 SOL created before this change can no longer be split or re-delegated. They must be deactivated and withdrawn first. The `GetMinimumDelegation` instruction returns the new value via return data, allowing on-chain programs to query it.

Rent Sysvar

The stored field `meta.rent_exempt_reserve` is deprecated. It is no longer read by any instruction handler and is unconditionally set to `PSEUDO_RENT_EXEMPT_RESERVE = 2,282,880` (the historical rent-exempt reserve for a stake account). All rent calculations now use `Rent::get()`, a syscall returning the current rent parameters from the runtime. This decouples the program from per-account historical rent values and prepares it for future rent changes. The `PSEUDO_RENT_EXEMPT_RESERVE` value is retained in the on-chain data purely for backwards compatibility with downstream programs that may read this field.

Optional Sysvar Accounts

The program previously required sysvar accounts (Clock, StakeHistory, Rent) to be passed as instruction accounts on some instructions. These are now obtained via syscalls. To maintain backwards compatibility, each instruction uses a diverge/converge pattern: it checks whether the first post-invariant account is a known sysvar. If so, the legacy account layout is consumed. If not, the new layout without sysvar accounts is assumed and the authority account is expected at that position with an explicit signer check. Both paths converge on the same downstream logic. The new interface enforces stricter positional signer checks on authority accounts, while the legacy interface retains the loose signer collection from the original native program for backwards compatibility.

Rewrite of the Split Instruction

The `Split` instruction is rewritten from a single code path using `validate_split_amount` and `Stake::split` into three explicit paths:

1. **Full split** (`split_lamports == source_lamport_balance`): The source account is closed and all state is copied to the destination. Delegation is preserved as-is, not recalculated from lamports. This path is intentionally lenient about the source state. Only the destination is validated against current rent and minimum delegation.
2. **Inactive split** (`!is_active_or_activating`): For fully deactivated stakes, initialized accounts, and uninitialized accounts. The source state is copied to the destination. Both accounts are checked for rent-exemption. No delegation math is performed, as the stake is not active. The source account's state is not re-serialized only the destination is written, but we did not find any security relevant bug with this.
3. **Active partial split**: For activating or active stakes. The destination must be pre-funded with at least one rent-exempt reserve. Delegation is redistributed with the split lamports: `source.delegation -= split_lamports` and `dest.delegation = split_lamports`. Both sides are validated against the minimum delegation. A sanity check verifies the source retains enough non-delegation lamports for rent.

Rent Change Interactions

Rent Decrease

A rent decrease is the planned direction (SIMD-0436/0437). When rent decreases, `Rent::get()` returns a smaller value than what existing accounts were funded with. The difference `old_rent - new_rent` becomes additional free lamports on every existing account. All operations were verified to work correctly under rent decrease: split, withdraw, delegate, merge, and move operations all become more permissive. The freed rent lamports can be withdrawn, re-delegated, or left as free balance. No invariant violations occur.

Rent Increase

SIMD-0392 proposes a mechanism that allows rent to increase, capped at the legacy rent-per-byte rate. The runtime-level grandfathering in SIMD-0392 (`min_allowed_balance = min(calculate_min_balance, pre_exec_balance)`) does not protect the stake program's internal invariant, because stake account lamport balances always exceed the rent minimum (1+ SOL delegation vs 0.002 SOL rent). The `min()` clause never activates for stake accounts.

If rent were to increase above what an account was funded with, every pre-existing active stake account would have its internal invariant violated: `delegation.stake + Rent::get() > lamports`. This invariant violation is invisible to the runtime, which only checks total balance against rent.

Anza is aware of this limitation.

4 | Findings

The change review did not reveal any security relevant findings.

We additionally added two fuzzcases to our on-chain fuzzing framework [Riverguard](#): a differential fuzzcase, which checks for unknown and undetected changes in the contract logic by simulating the same transactions on both versions of the stake program, and another fuzzcase, which checks which programs depend on `meta.rent_exempt_reserve` by modifying the field before re-simulating the instruction.

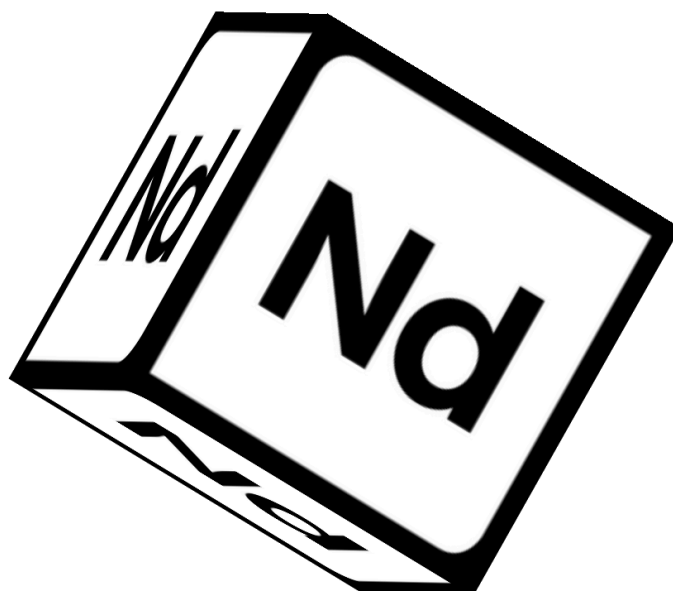
A | About Neodyme

Security is difficult.

To understand and break complex things, you need a certain type of people. People who thrive in complexity, who love to play around with code, and who don't stop exploring until they fully understand every aspect of it. That's us.

Our team never outsources audits. Having found over 80 High or Critical bugs in Solana's core code itself, we believe that Neodyme hosts the most qualified auditors for Solana programs. We've also found and disclosed critical vulnerabilities in many of Solana's top projects and have responsibly disclosed issues that could have resulted in the theft of over \$10B in TVL on the Solana blockchain.

All of our team members have a background in competitive hacking. During such hacking competitions, called CTFs, we competed and collaborated while finding vulnerabilities, breaking encryption, reverse engineering complicated algorithms, and much more. Through the years, many of our team members have won national and international hacking competitions, and keep ranking highly among some of the hardest CTF events worldwide. In 2020, some of our members started experimenting with validators and became active members in the early Solana community. With the prospect of an interesting technical challenge and bug bounties, they quickly encouraged others from our CTF team to look for security issues in Solana. The result was so successful that after reporting several bugs, in 2021, the Solana Foundation contracted us for source code auditing. As a result, Neodyme was born.



B | Methodology

We are not checklist auditors.

In fact, we pride ourselves on that. We adapt our approach to each audit, investing considerable time into understanding the program upfront and exploring its expected behavior, edge cases, invariants, and ways in which the latter could be violated.

We use our uniquely deep knowledge of Solana internals, and our years-long experience in auditing Solana programs to find bugs that others miss. We often extend our audit to cover off-chain components in order to see how users could be tricked or the contract affected by bugs in those components.

Nonetheless, we also have a list of common vulnerability classes, which we always exhaustively look for. We provide a sample of this list here.

Select Common Vulnerabilities

Our most common findings are still specific to Solana itself. Among these are vulnerabilities such as the ones listed below:

- Insufficient validation, such as:
 - Missing ownership checks
 - Missing signer checks
 - Signed invocation of unverified programs
 - Account confusions
 - Missing freeze authority checks
 - Insufficient SPL account verification
 - Dangerous user-controlled bumps
 - Insufficient Anchor account linkage
- Account reinitialization vulnerabilities
- Account creation DoS
- Redeployment with cross-instance confusion
- Missing rent exemption assertion
- Casting truncation
- Arithmetic over- or underflows
- Numerical precision and rounding errors
- Anchor pitfalls, such as accounts not being reloaded
- Non-unique seeds
- Issues arising from CPI recursion
- Log truncation vulnerabilities
- Vulnerabilities specific to integration of Token Extensions, for example unexpected external token hook calls

Apart from such Solana-specific findings, some of the most common vulnerabilities relate to the general logical structure of the contract. Specifically, such findings may be:

- Errors in business logic
- Mismatches between contract logic and project specifications
- General denial-of-service attacks
- Sybil attacks
- Incorrect usage of on-chain randomness
- Contract-specific low-level vulnerabilities, such as incorrect account memory management
- Vulnerability to economic attacks
- Allowing front-running or sandwiching attacks

Miscellaneous other findings are also routinely checked for, among them:

- Unsafe design decisions that might lead to vulnerabilities being introduced in the future
 - Additionally, any findings related to code consistency and cleanliness
- Rug pull mechanisms or hidden backdoors

Often, we also examine the authority structure of a contract, investigating their security as well as the impact on contract operations should they be compromised.

Over the years, we have found hundreds of high and critical severity findings, many of which are highly nontrivial and do not fall into the strict categories above. This is why our approach has always gone way beyond simply checking for common vulnerabilities. We believe that the only way to truly secure a program is a deep and tailored exploration that covers all aspects of a program, from small low-level bugs to complex logical vulnerabilities.

C | Vulnerability Severity Rating

We use the following guideline to classify the severity of vulnerabilities. Note that we assess each vulnerability on an individual basis and may deviate from these guidelines in cases where it is well-founded. In such cases, we always provide an explanation.

Severity	Description
CRITICAL	Vulnerabilities that will likely cause loss of funds. An attacker can trigger them with little or no preparation, or they are expected to happen accidentally. Effects are difficult to undo after they are detected.
HIGH	Bugs that can be used to set up loss of funds in a more limited capacity, or to render the contract unusable.
MEDIUM	Bugs that do not cause direct loss of funds but that may lead to other exploitable mechanisms, or that could be exploited to render the contract partially unusable.
LOW	Bugs that do not have a significant immediate impact and could be fixed easily after detection.
INFORMATIONAL	Bugs or inconsistencies that have little to no security impact, but are still noteworthy.

Additionally, we often provide the client with a list of nit-picks, i.e. findings whose severity lies below Informational. In general, these findings are not part of the report.

Neodyme AG

Dirnismaning 55
Halle 13
85748 Garching
Germany

E-Mail: contact@neodyme.io

<https://neodyme.io>